MŰEGYETEM 1782

Mesterséges Intelligencia Alapjai

BMEGEGTBX01

Forgácsolási kísérletek adatainak felhasználása szerszámállapot felügyeletre MI módszerek alkalmazásával

Szalai Zsolt
I6JTV3
2025. 11. 23

Tartalom

1. Bevezetés ... 3
2. Adatforrás és kísérleti háttér ... 3
3. Adatelőkészítés és Feature Engineering ... 4
    3.1. Szűrés és Hibakezelés ... 4
    3.2. Adatstruktúrák (2D vs 3D) ... 5
    3.3. Validációs Stratégia: A 80/20-as szabály és a csoportos felosztás ... 5
4. Alapgépi tanulási modellek (Baseline) ... 6
    4.1. Logistic Regression ... 6
    4.2. Random Forest ... 7
5. Időszoros Neurális Háló (Deep Learning) ... 8
    5.1. Architektúra (1D-CNN) ... 8
    5.2. Eredmények ... 9
6. Összegzés ... 10
7. Mellékletek és Forráskód ... 11
8. Függelék: Telepítési és Futtatási Útmutató ... 11
    8.1 Előkészületek ... 11
    8.2 Környezet telepítése ... 11
    8.3 A Szkriptek Futtatása ... 11

1. Bevezetés

A modern gyártástechnológiai rendszerekben a forgácsolási folyamatok megbízhatósága kiemelt jelentőségű. A szerszámkopás időben történő felismerése közvetlen hatással van a gyártási minőségre, a selejtarányra és a gép rendelkezésre állására. A hagyományos szerszámállapot-felügyeleti módszerek gyakran szakemberi jelenlétet vagy költséges infrastruktúrát igényelnek.

A gépi tanulás (Machine Learning, ML) és a mélytanulás (Deep Learning, DL) lehetőséget biztosít arra, hogy a CNC gépekről származó szenzoradatok alapján automatizált kopásfelügyeletet építsünk ki. A projekt célja a University of Michigan SMART Lab által közölt CNC Milling dataset felhasználásával egy MI-alapú rendszer kialakítása.

A projekt összehasonlítja a klasszikus „pillanatkép-alapú" (Logistic Regression, Random Forest) modelleket a fejlett, idősoros elemzést végző konvolúciós neurális hálókkal (1D-CNN). A fejlesztés Python nyelven, moduláris kódszerkezettel készült.

2. Adatforrás és kísérleti háttér

A felhasznált adathalmaz a Michigan SMART Lab "CNC Milling Machine Tool Wear" *[1]* adatgyűjtési sorozatából származik. A kísérletek során 18 különféle forgácsolási műveletet rögzítettek 100 ms mintavételezéssel.

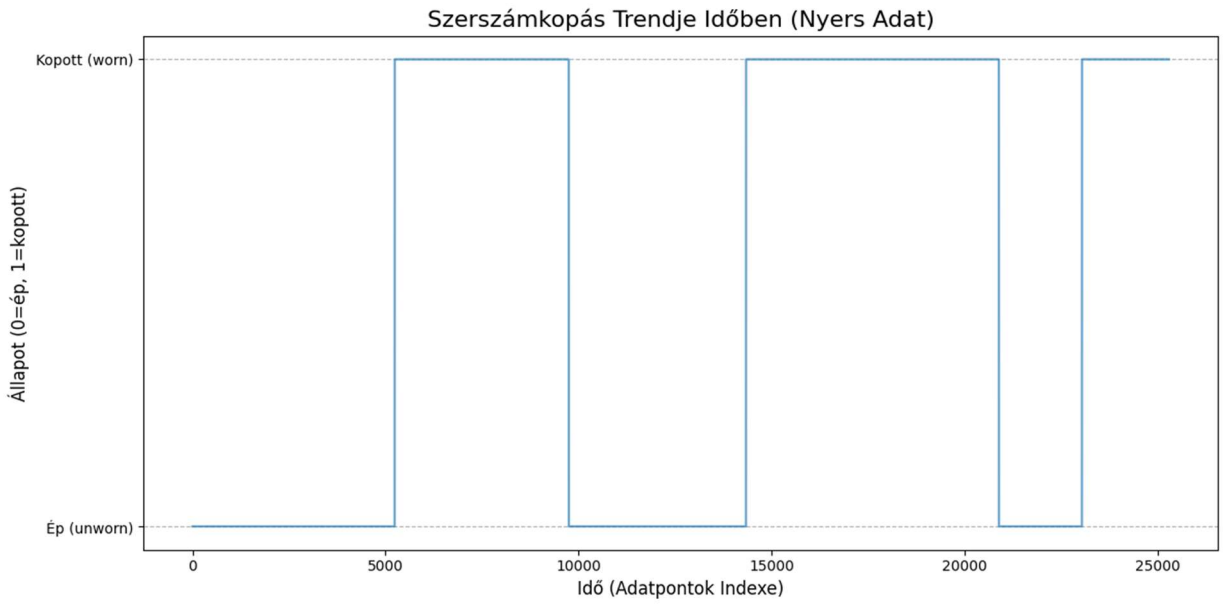
Adatintegráció (*src/merge_experiments.py*): Az eredeti adathalmaz 18 különálló fájlban (*experiment_01.csv ... 18.csv*) tartalmazta a szenzor adatokat, míg a címkék (tool condition) egy külön train.csv fájlban voltak. A feldolgozás nulladik lépéseként egy saját Python szkripttel (merge_experiments.py) egyesítettem ezeket a forrásokat. A szkript összefésülte az idősoros adatokat a megfelelő címkékkel, így jött létre a projekt bemeneteként szolgáló egységes ***merged_data.csv***.

Megjegyzés: Ez az állomány már generálva rendelkezésre áll a data/ mappában, így az egyesítő szkript futtatása a kiértékeléshez nem szükséges, de a reprodukálhatóság érdekében mellékeltem.

A rendszer a következőket figyelte:

- X, Y, Z tengelyek és a főorsó aktuális pozíciója, sebessége, gyorsulása.
- Áramfelvétel, teljesítmény, kimeneti áram/feszültség.
- A megmunkálási lépés típusa (Machining_Process).
- A szerszám állapota („worn" vagy „unworn") azaz kopott és ép.

A felhasznált adathalmaz 18 forgácsolási kísérlet szenzoradatait tartalmazza. Az adatok vizualizációja során jól látható, hogyan váltakoznak a kísérletek a kopott és ép szerszámok között az idő függvényében.



1. ábra: A szerszámállapot (kopott/ép) alakulása a teljes, nyers adathalmazon az idő függvényében.

3. Adatelőkészítés és Feature Engineering

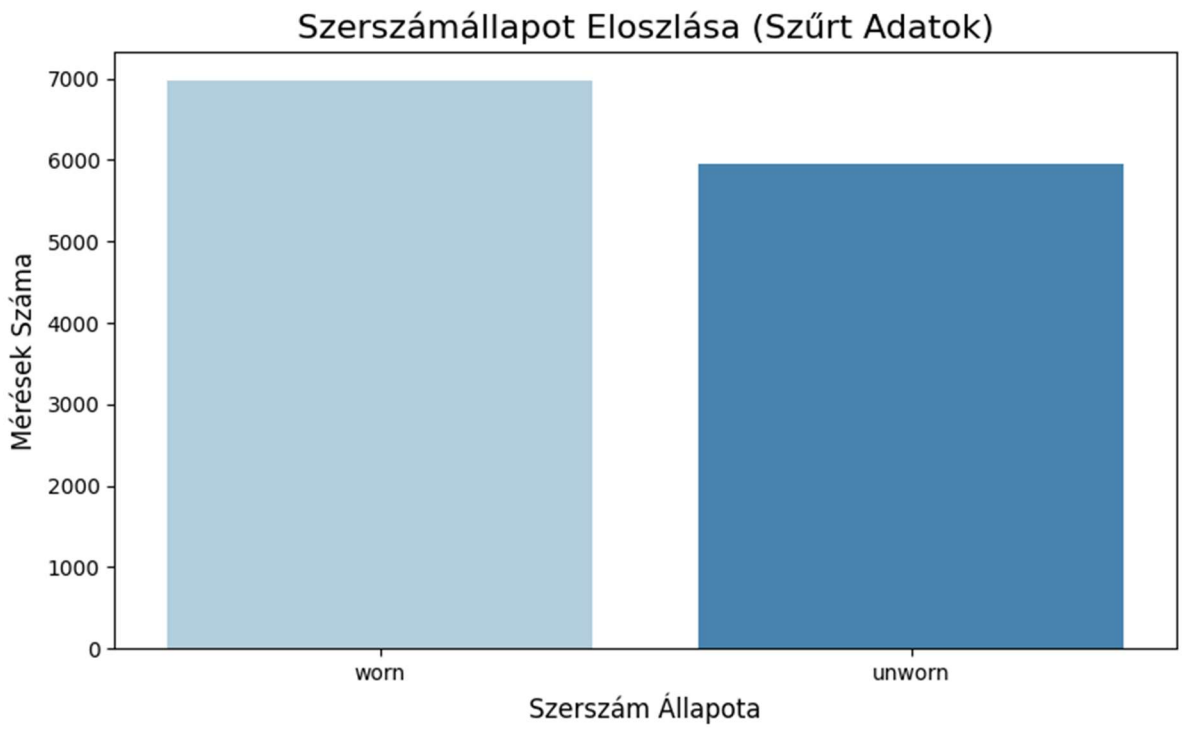
A *features.py* modul végzi a nyers adatok tisztítását és transzformációját.

3.1. Szűrés és Hibakezelés

A projekt során azonosításra került, hogy az eredeti dokumentációban javasolt szűrési feltétel (Program Number == 0) félrevezető, mivel az nagy mennyiségű adat elvesztését eredményezte. A korrigált logika alapján a rendszer:

1. Eltávolítja a hibás szenzorértékeket (Feedrate != 50, Position != 198).
2. A *Machining_Process* oszlop alapján kizárólag a 6 aktív forgácsolási fázist („Layer 1–3 Up/Down”) tartja meg. Eredményként **12 943 tiszta adatpont** állt rendelkezésre.

Fontos volt ellenőrizni, hogy a szűrés után az osztályok (worn/unworn) egyensúlya megfelelő-e a tanításhoz.



2. ábra: A tanító adatok eloszlása a szűrés után. Látható, hogy mindkét osztályból (worn/unworn) elegendő minta áll rendelkezésre.

3.2. Adatstruktúrák (2D vs 3D)

A modul kétféle működési módot támogat (mode paraméter):

- **Snapshot mód (2D):** A klasszikus modellekhez minden 100 ms-os mérést független adatpontként kezel.
- **Time-series mód (3D):** A neurális hálóhoz **csúszó ablakos (sliding window)** technikát alkalmaz. 50 időlépés (5 másodperc) alkot egy mintát, így a modell képes a folyamat időbeli változásait (trendjeit) is értelmezni.

3.3. Validációs Stratégia: A 80/20-as szabály és a csoportos felosztás

A gépi tanulási modellek kiértékelésénél kritikus fontosságú a rendelkezésre álló adatok szétválasztása tanító (training) és tesztelő (test) halmazokra. A projekt során a klasszikus **80% (tanító) – 20% (teszt)** arányt alkalmaztuk.

Azonban az idősoros adatoknál a hagyományos, véletlenszerű soronkénti szétválasztás (random shuffle) súlyos módszertani hibához, úgynevezett **Adatszivárgáshoz (Data Leakage)** vezetett volna.

- *A probléma:* Ha az 50 lépéses ablakokat véletlenszerűen keverjük, a tanító és a teszt halmazba is kerülhetnek egymást átfedő, szinte azonos ablakok. Így a modell nem "tanul", hanem "emlékszik".
- *A megoldás (Group Split):* A felosztást nem az ablakok, hanem a **kísérletek (Experiment ID)** szintjén végeztük el.
    - A 18 kísérletből véletlenszerűen kiválasztottunk 14-et a tanításhoz (kb. 80%).

- A maradék 4 kísérletet teljesen elkülönítettük a teszteléshez (kb. 20%).

Ez biztosítja a modell valódi **generalizációs képességének** mérését: a teszt során olyan gépi állapotokkal találkozik, amelyeket a tanítás során soha, semmilyen formában nem látott.

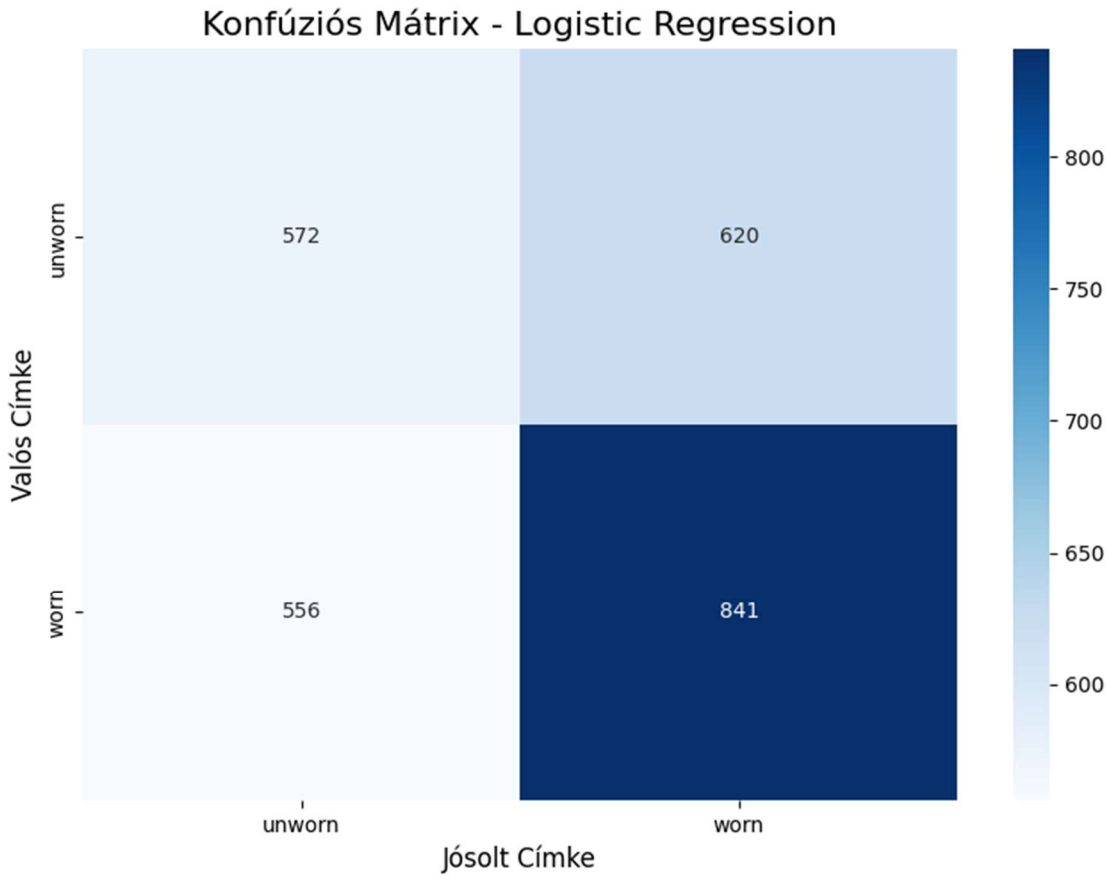
4. Alapgépi tanulási modellek (Baseline)

A *baseline_models.py* a statikus (2D) adatokon vizsgálta a predikciós képességet:

4.1. Logistic Regression

- StandardScaler és Pipeline alkalmazásával.
- **Eredmény:** ~55% pontosság.
- **Következtetés:** A szenzoradatok és a kopás közötti kapcsolat nem lineáris, így ez a modell nem elégséges.

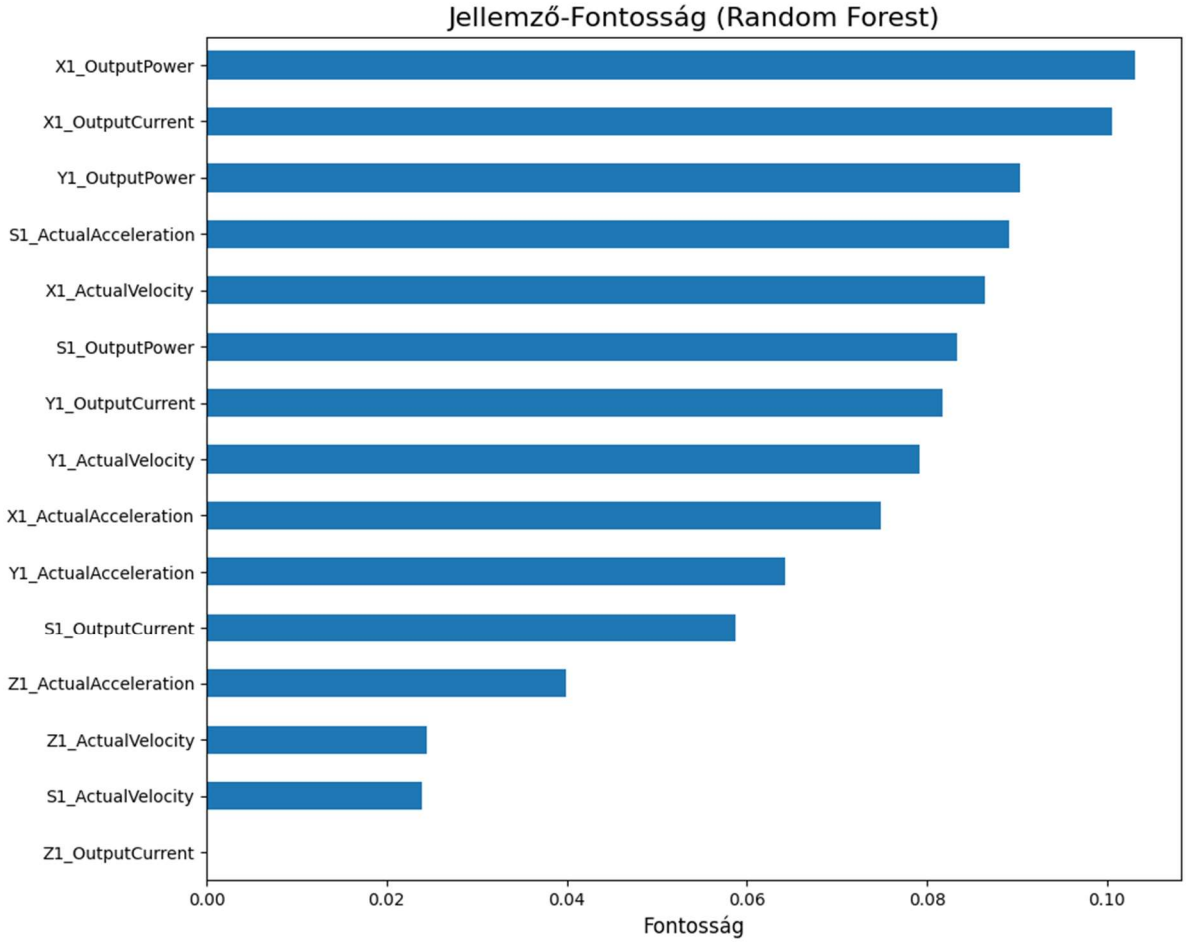
A Logistic Regression modell teljesítményét a konfúziós mátrixszal értékeltük:



3. ábra: A Logistic Regression modell konfúziós mátrixa. A ~55%-os pontosság mellett látható a tévesztések száma a mellékátlón.

4.2. Random Forest

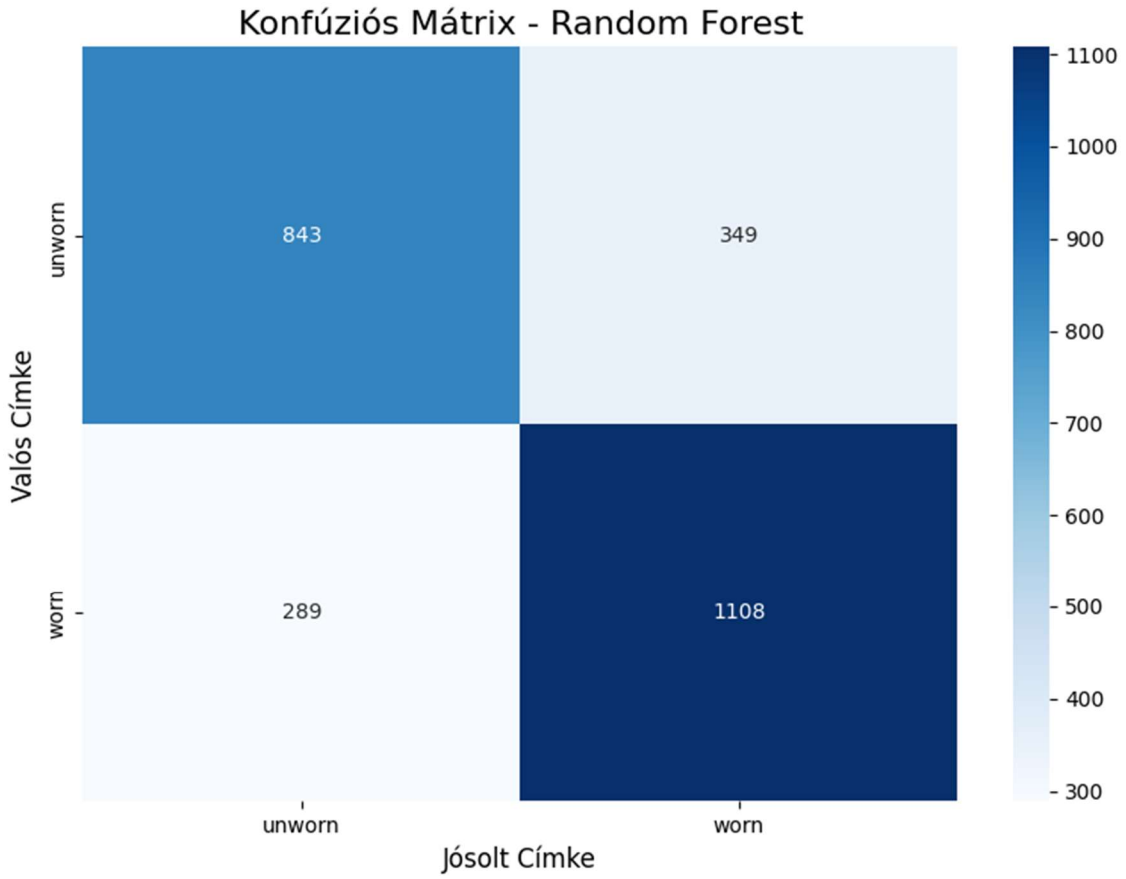
Referenciaként egy Random Forest klasszifikátort alkalmaztunk. A modell egyik előnye, hogy megmutatja, mely szenzorok játszották a legnagyobb szerepet a döntésben.



4. ábra: Jellemző-fontosság (Feature Importance). A diagram mutatja, mely szenzoradatok (pl. teljesítmény, áramfelvétel) voltak a legmeghatározóbbak a kopás felismerésében.

- Nem igényelt skálázást, robusztus a zajos adatokkal szemben.
- **Eredmény:** ~75% pontosság.
- **Következtetés:** Ez lett a projekt alapvonala (baseline). Bizonyította, hogy a pillanatnyi adatokban is van információ, de a megbízható detektáláshoz kevés.

A Random Forest modell teljesítményét a konfúziós mátrixszal értékeltük:



5. ábra: A Random Forest modell konfúziós mátrixa. A ~75%-os pontosság mellett látható a tévesztések száma a mellékátlón.

5. Idősoros Neurális Háló (Deep Learning)

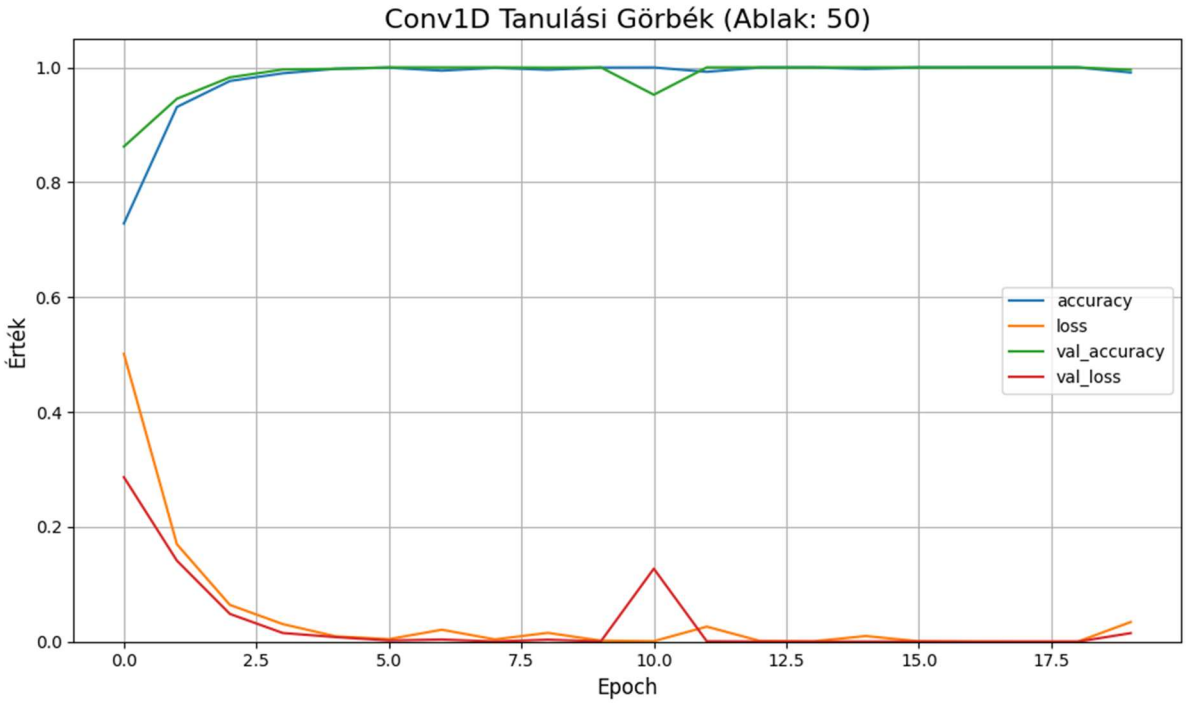
A 75%-os korlát áttörésére a rendszer kiegészült a *nn_model_timeseries.py* modullal.

5.1. Architektúra (1D-CNN)

A modell konvolúciós rétegeket alkalmaz az időbeli mintázatok felismerésére:

- **Input:** (50 időlépés, 15 szenzor).
- **Rétegek:** 2 db Conv1D + MaxPooling1D a jellemzők kinyerésére, Flatten, majd Dense rétegek a klasszifikációhoz.
- **Regularizáció:** Dropout a túltanulás elkerülésére.

A projekt fő eredménye a Konvolúciós Neurális Háló. A modell tanítása során a pontosság (accuracy) gyorsan növekedett, a veszteség (loss) pedig stabilan csökkent, ami a sikeres konvergencia jele.



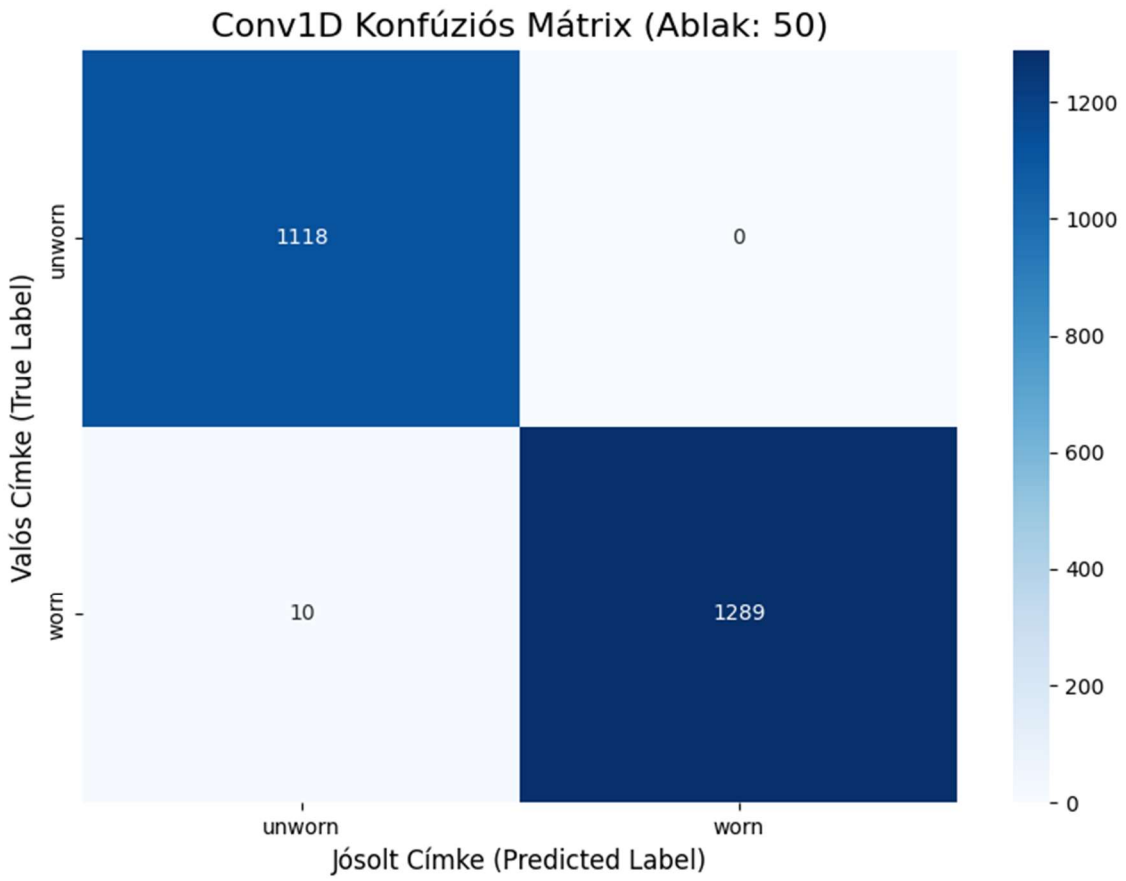
6. ábra: A Conv1D modell tanulási görbéi. A validációs pontosság már a korai epoch-okban megközelíti a 100%-ot.

5.2. Eredmények

A modell drámai javulást hozott a baseline-hoz képest:

- **Pontosság: ~99.6%** (Teszt halmazon).
- **Hiba-elemzés:** A konfúziós mátrix alapján a modell 0 téves riasztást (False Positive) produkált, és mindössze 10 esetben nem vette észre a kopást a több mint 2400 tesztesetből.

A konvolúciós neurális háló teljesítményét szintén konfúziós mátrixszal értékeltük:



7. ábra: A Conv1D modell konfúziós mátrixa. A mátrix igazolja a 99.6%-os pontosságot: a tévesztések száma minimális (0 False Positive, 10 False Negative).

6. Összegzés

A különböző modellek számítási igénye. Mérések:

- **Random Forest/Logistic Regression:** A tanítási idő rendkívül rövid (< 5 másodperc), mivel a modell nem igényel iteratív optimalizációt (epoch-okat). A memóriaigénye alacsony.
- **Conv1D Neurális Háló:** A tanítási idő jelentősen hosszabb (kb. 2-3 perc / 20 epoch CPU-n). Az adat-előkészítés (3D ablakozás) memóriaigénye is többszöröse a statikus módszernek (a redundáns adatok miatt). **Tapasztalat:** Bár a neurális háló számítási igénye nagyobb, a pontosságban elért 25 százalékpontos javulás (~75% -> ~100%) ipari környezetben indokolja a többlet erőforrás használatát. Valós idejű (inference) futtatáskor a Conv1D modell válaszideje (<50ms) továbbra is bőven a gyártási ciklusidőn belül marad.

A projekt sikeresen demonstrálta, hogy a szerszámkopás-felügyelet hatékonysága nagyságrendekkel növelhető, ha a statikus elemzés helyett idősoros (time-series) megközelítést alkalmazunk. Az elkészült rendszer a nyers adatok betöltésétől a neurális háló kiértékeléséig egy teljes, automatizált pipeline-t alkot.

7. Mellékletek és Forráskód

Felhasznált adatsorozat: *[1]*: https://www.kaggle.com/datasets/shasun/tool-wear-detection-in-cnc-mill

A projekt átadott struktúrája:

- *src/features.py*: Adatfeldolgozás, 2D/3D transzformáció, vizualizáció.
- *src/baseline_models.py*: Klasszikus modellek (LR, RF) futtatása.
- *src/nn_model_timeseries.py*: Idősoros CNN modell futtatása.
- *merged_data.csv*: Bemeneti adatfájl.
- *dataREADME.md*: Adatsor részletes használati útmutatója
- src/merge_experiments.py: A nyers kísérleti fájlok és a címkék egyesítése
- *requirements.txt:* Futtatáshoz szükséges könyvtárak

8. Függelék: Telepítési és Futtatási Útmutató

A projekt futtatásához Python 3.8+ környezet szükséges. A forráskód és a szükséges állományok a mellékelt ZIP fájlban találhatóak.

8.1 Előkészületek

1. Csomagolja ki a ZIP állományt egy tetszőleges mappába.
2. Győződjön meg róla, hogy a *merged_data.csv* fájl a *data/CNC_Milling_Data/* mappában található.
3. Nyisson egy terminált a projekt főkönyvtárában.

8.2 Környezet telepítése

A szükséges Python könyvtárak (pandas, tensorflow, scikit-learn, stb.) telepítése a mellékelt *requirements.txt* fájl segítségével történik:

```
pip install -r requirements.txt
```

8.3 A Szkriptek Futtatása

A rendszer moduláris, az egyes részek külön-külön is futtathatóak a src mappából hivatkozva:

0. Lépés: Adatok Egyesítése (OPCIONÁLIS / MÁR ELVÉGEZVE) A *src/merge_experiments.py* felelős a nyers *experiment_XX.csv* fájlok egyesítéséért.

- **Státusz:** Lefuttatva. A kimeneti *merged_data.csv* már ott van a *data/* mappában.
- **Teendő: Nincs.** (Csak akkor futtassa, ha a nyers forrásfájlokból újra akarja generálni az adatsort).

A) Baseline Modellek (Klasszikus ML) futtatása: Ez a szkript betölti a 2D-s adatokat, lefuttatja a Logisztikus Regressziót és a Random Forest-et, majd legenerálja a kapcsolódó ábrákat.

```
python src/baseline_models.py
```

B) Neurális Háló (Conv1D) futtatása: Ez a szkript végzi az idősoros (sliding window) adatátalakítást, a modell tanítását, és a végső 99.6%-os pontosságú kiértékelést.

```
python src/nn_model_timeseries.py
```

Megjegyzés: A szkriptek futás közben a főkönyvtárba mentik a generált ábrákat (PNG formátumban).